🎬 Multi-Scene Whiteboard Video Pipeline

Creates **full-length whiteboard animation videos** (1-10+ minutes) with:

- Multiple scenes/frames, each with their own illustrations
- Smooth transitions between scenes (erase/wipe effect)
- Full narration synced to every component across all scenes
- Stroke-by-stroke hand-drawing animation with pen cursor

Architecture

```
TOPIC → LLM plans N scenes → Generate N frames → Full narration audio
      → Build sync timeline across all scenes → Render → Final MP4
```

Step 1: Setup

```
!pip install -q google-genai elevenlabs opencv-python-headless scikit-image scipy numpy Pillow requests

import cv2
import numpy as np
import json
import os
import time
import requests
import base64
from PIL import Image, ImageDraw
from skimage.morphology import skeletonize
from scipy.spatial import KDTree
from matplotlib import pyplot as plt
from IPython.display import HTML, Audio, display
from base64 import b64encode
from google.colab import userdata

print("✅ All packages installed")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 20.3 MB/s eta 0:00:00
✅ All packages installed
```

```
# ============================================================
# 🔑 API KEYS
# Option 1: Set them directly here
# Option 2: Use Colab Secrets (recommended)
#   → Click 🔑 icon in left sidebar → Add secrets
# ============================================================

# Direct input (uncomment and paste your keys):
GEMINI_API_KEY = "AIzaSyCRFMtukjWcSrVovU9aCquazN3kwVmDllI"
ELEVENLABS_API_KEY = "sk_1dd965f7e18bb81ffe6a25183b327117798ee362acd002b1"

# Initialize Gemini
from google import genai
from google.genai import types
gemini_client = genai.Client(api_key=GEMINI_API_KEY)
print("✅ Gemini client ready")
```

```
✅ Gemini client ready
```

Step 2: Topic & Video Settings

```
# ============================================================
# 📝 CONFIGURE YOUR VIDEO
# ============================================================

TOPIC = """LA is the most posterior part of the heart A ; LA
enlargement (eg, in mitral stenosis) can lead to:
Compression of esophagus dysphagia
Compression of left recurrent laryngeal
nerve (branch of vagus nerve hoarseness
(Ortner syndrome)
RV is the most anterior part of the heart and
```

```
most commonly injured in trauma. LV is
about 2/3 and RV is about 1/3 of the inferior
(diaphragmatic) cardiac surface B.
B
SVC
Azygos vein
IVC
RA
RV
LA
LV
Aortic knob
Pulmonary artery
Descending aorta
Pericardium Consists of 3 layers (from outer to inner):
Fibrous pericardium
Parietal pericardium
Epicardium (visceral pericardium)
Pericardial space lies between parietal
pericardium and epicardium.
Pericardium innervated by phrenic nerve.
Pericarditis can cause referred pain to the neck,
arms, or one or both shoulders (often left).
Fibrous pericardium
Parietal pericardium
Pericardial space
Epicardium (visceral pericardium)
Coronary vessels
Myocardium
Endocardium
Coronary blood
supply
RCA
Key:
AMA = Acute marginal artery
= Left anterior descending artery
LAD LCA
(or LM) = Left (main) coronary artery
LCX = Left circumflex artery
OMA = Obtuse marginal artery
PDA = Posterior descending artery
PT = Pulmonary trunk
PV = Pulmonary vein
RCA = Right coronary artery
LAD and its branches supply anterior 2/3 of
interventricular septum, anterolateral papillary
muscle, and anterior surface of LV. Most
commonly occluded.
Posterior descending artery (PDA) supplies
posterior 1/3 of interventricular septum,
posterior 2/3 of ventricular walls, posteromedial
papillary muscle, and SA and AV nodes (as
determined by dominance). Infarct may cause
nodal dysfunction (bradycardia or heart block).
Right (acute) marginal artery supplies RV.
Dominance:
Right-dominant circulation (most common)
= PDA arises from RCA
Left-dominant circulation = PDA arises
from LCX (~5%–10% of patients)
Codominant circulation = PDA arises from
both LCX and RCA (~10%–20% of patients)
Coronary blood flow to LV and interventricular
septum peaks in early diastole.
Coronary sinus runs in the left AV groove and
drains into the RA

"""

NUM_SCENES = 4                    # Number of frames/scenes (each ~15-30s)
COMPONENTS_PER_SCENE = 4          # Sections per frame (2-6)
VOICE_ID = "21m00Tcm4TlvDq8ikWAM" # ElevenLabs voice (Rachel)

# Video settings
FPS = 30
REVEAL_RADIUS = 90
REVEAL_STYLE = 'skeleton'    # 'skeleton', 'outline_fill', or 'expand'
SHOW_CURSOR = False
TRANSITION_DURATION = 1.0         # Seconds for scene transition
TRANSITION_STYLE = 'erase'        # 'erase', 'fade', 'instant'

# Estimated duration
est_per_scene = COMPONENTS_PER_SCENE * 7  # ~7s per component
```

```
est_per_scene = COMPONENTS_PER_SCENE * 7  # ~7s per component
est_total = NUM_SCENES * est_per_scene
print(f"Topic: {TOPIC}")
print(f"Scenes: {NUM_SCENES} × {COMPONENTS_PER_SCENE} components")
print(f"Estimated duration: ~{est_total//60}m {est_total%60}s")
print(f"Transition: {TRANSITION_STYLE}")
```

```
Topic: LA is the most posterior part of the heart A ; LA
enlargement (eg, in mitral stenosis) can lead to:
Compression of esophagus dysphagia
Compression of left recurrent laryngeal
nerve (branch of vagus nerve hoarseness
(Ortner syndrome)
RV is the most anterior part of the heart and
most commonly injured in trauma. LV is
about 2/3 and RV is about 1/3 of the inferior
(diaphragmatic) cardiac surface B.
B
SVC
Azygos vein
IVC
RA
RV
LA
LV
Aortic knob
Pulmonary artery
Descending aorta
Pericardium Consists of 3 layers (from outer to inner):
Fibrous pericardium
Parietal pericardium
Epicardium (visceral pericardium)
Pericardial space lies between parietal
pericardium and epicardium.
Pericardium innervated by phrenic nerve.
Pericarditis can cause referred pain to the neck,
arms, or one or both shoulders (often left).
Fibrous pericardium
Parietal pericardium
Pericardial space
Epicardium (visceral pericardium)
Coronary vessels
Myocardium
Endocardium
Coronary blood
supply
RCA
Key:
AMA = Acute marginal artery
= Left anterior descending artery
LAD LCA
(or LM) = Left (main) coronary artery
LCX = Left circumflex artery
OMA = Obtuse marginal artery
PDA = Posterior descending artery
PT = Pulmonary trunk
PV = Pulmonary vein
RCA = Right coronary artery
LAD and its branches supply anterior 2/3 of
interventricular septum, anterolateral papillary
muscle, and anterior surface of LV. Most
commonly occluded.
Posterior descending artery (PDA) supplies
posterior 1/3 of interventricular septum,
posterior 2/3 of ventricular walls, posteromedial
```

Step 3: Generate Multi-Scene Script (LLM)

The LLM plans the ENTIRE video: all scenes, all narration, all image prompts, all sync points.

```
MULTI_SCENE_PROMPT = f"""You are a whiteboard animation video planner for a YouTube medical/educational channel.
Create a complete multi-scene whiteboard video plan in the style of popular YouTube whiteboard explainers
(like Ninja Nerd, Med School Insiders, Osmosis).

TOPIC: {TOPIC}
NUMBER OF SCENES: {NUM_SCENES}
COMPONENTS PER SCENE: {COMPONENTS_PER_SCENE}

Respond with ONLY valid JSON (no markdown, no backticks):

{{
  "title": "Video title",
  "total_scenes": {NUM_SCENES},

  "scenes": [
    {{
```

```
    ``
      "scene_number": 1,
      "scene_title": "Short descriptive title for this scene",

      "narration": "Complete narration for this scene only. Must have exactly {COMPONENTS_PER_SCENE} sentences, o

      "sync_map": [
        {{
          "component": 1,
          "narration_segment": "Exact sentence from narration for this component",
          "trigger_phrase": "2-4 distinctive words from START of this segment"
        }}
      ],

      "layout_style": "Name of the layout chosen for this scene",

      "image_prompt": "Detailed prompt following ALL rules below"
    }}
  ]
}}

=== LAYOUT STYLES (assign a DIFFERENT one to each scene) ===

For {COMPONENTS_PER_SCENE} components, choose from these layouts:

LAYOUT A - "big_left_stack_right": One large component on the left half, three smaller ones stacked vertically on

LAYOUT B - "top_banner_three_below": One wide component spanning the full top, three equal components in a row be

LAYOUT C - "center_focus": One large component in the center, three smaller ones around it (top-left, top-right, l

LAYOUT D - "timeline_horizontal": Four components in a single horizontal row like a process flow.

LAYOUT E - "diagonal_cascade": Components placed in a diagonal staircase pattern from top-left to bottom-right.

LAYOUT F - "inverted_L": One tall component on the left side, one wide component across the top-right, two square

LAYOUT G - "pyramid": One component centered at top, two components in the middle row, one wide component at the l

LAYOUT H - "asymmetric_grid": Two large components on top (one wider than the other), two on the bottom (reversed

IMPORTANT: Every scene MUST use a DIFFERENT layout. Do NOT repeat layout styles across scenes.

=== IMAGE PROMPT RULES (include ALL of these in EVERY image_prompt) ===

VISUAL STYLE - "YouTube Whiteboard Explainer" (like Ninja Nerd, Osmosis, Med School Insiders):
This is the most important instruction. The image must look like a frame from a popular YouTube whiteboard animat

BACKGROUND:
- Warm CREAM / OFF-WHITE background color (#F5F0E8 or similar warm light beige)
- NOT pure white - it should feel warm and slightly textured like a real whiteboard or paper

MAGENTA BORDER RULES (for component detection - these get removed later):
- Each section is enclosed in a BRIGHT MAGENTA (#FF00FF) SOLID THICK RECTANGLE border (at least 6 pixels thick)
- Borders must be STRAIGHT LINES only, perfectly rectangular
- MAGENTA color is used ONLY for borders and NOWHERE else in the entire image
- There must be at least 30 pixels of CREAM GAP between any two bordered sections
- There must be EXACTLY {COMPONENTS_PER_SCENE} magenta-bordered sections, no more, no less
- NO content may touch, overlap, or cross ANY magenta border
- ALL drawn content must stay FULLY INSIDE its bordered section with at least 15 pixels margin
- Arrows or connectors BETWEEN sections are strictly FORBIDDEN
- Nothing exists OUTSIDE the bordered sections except cream background

TEXT STYLE:
- Section titles/headings in BOLD BLACK hand-lettered style, large and prominent (this is the main visual element
- Important/emphasis words in BOLD RED or RED with red underline
- Sub-text and labels in smaller black handwriting
- Text should be the DOMINANT visual element - big, bold, readable
- Use ALL CAPS for main headings

ILLUSTRATION STYLE:
- Simple, clean, iconic illustrations NEXT TO or BELOW the bold text
- Use a RICH COLOR PALETTE: blue, red, green, yellow, orange, pink, black
- Colored underlines beneath key terms (pink, green, blue underlines)
- Simple graphs, charts, arrows, icons drawn in a hand-sketched style
- Illustrations are supplementary to the text, not the main focus
- Think: a whiteboard a professor draws on - text-heavy with supporting sketches

COLOR PALETTE:
- BLACK: main text, outlines, titles
- RED: emphasis text, important callouts, warning labels, ≠ signs
- BLUE: graphs, charts, upward trends, positive elements
- GREEN: underlines for good/positive terms, checkmarks
```

```
- YELLOW: highlighter marker illustrations, accent elements
- PINK: underlines for medical terms, emphasis
- ORANGE: pathogens, negative elements, warning icons
- NO magenta/fuchsia in illustrations (reserved for borders only)

COMPOSITION PER SECTION:
- Each section should have a LARGE BOLD TITLE (the main visual anchor)
- Below or beside the title: a simple supporting illustration or icon
- Below: a short KEY PHRASE or tagline in smaller text with colored underline
- The overall feel should be: "a still frame from a YouTube whiteboard explainer video"

CRITICAL REMINDERS:
- EXACTLY {COMPONENTS_PER_SCENE} bordered sections. Not more, not less.
- NOTHING touches or crosses a magenta border.
- NO arrows, lines, or decorations between sections.
- Background is CREAM/OFF-WHITE, not pure white.
- Text is the DOMINANT element, illustrations are supplementary.
- Use multiple colors freely (black, red, blue, green, yellow, pink, orange).

=== RULES ===
- Each scene narration must have EXACTLY {COMPONENTS_PER_SCENE} sentences
- Scenes flow logically like chapters
- Each trigger_phrase must be 2-4 UNIQUE words from the beginning of its segment
- EVERY scene MUST use a DIFFERENT layout_style from the list above
- Scene 1 introduces the topic, final scene concludes/summarizes
- image_prompt must describe the specific layout arrangement AND specific visual content of each section includin
"""

print(f"🧠 Planning {NUM_SCENES}-scene video...")

response = gemini_client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=MULTI_SCENE_PROMPT,
    config=types.GenerateContentConfig(
        temperature=0.7,
        max_output_tokens=8000,
    )
)

raw = response.text.strip()
if raw.startswith("```"):
    raw = raw.split("```")[1]
    if raw.startswith("json"):
        raw = raw[4:]
    raw = raw.strip()

video_plan = json.loads(raw)

# Display plan summary
print(f"\n🎬 VIDEO: {video_plan['title']}")
print(f"   Scenes: {len(video_plan['scenes'])}")
print("=" * 60)
for scene in video_plan['scenes']:
    print(f"\n📋 Scene {scene['scene_number']}: {scene['scene_title']}")
    print(f"   Layout: {scene.get('layout_style', 'not specified')}")
    print(f"   Narration: {scene['narration'][:80]}...")
    print(f"   Components: {len(scene['sync_map'])}")
    for s in scene['sync_map']:
        print(f"     #{s['component']}: [{s['trigger_phrase']}]")

print(f"\n✅ Video plan ready: {len(video_plan['scenes'])} scenes")
```

```
🧠 Planning 4-scene video...

🎬 VIDEO: Cardiac Anatomy and Coronary Circulation Essentials
   Scenes: 4
============================================================

📋 Scene 1: Heart Orientation and Surfaces
   Layout: LAYOUT A - big_left_stack_right
   Narration: The right ventricle forms the most anterior heart surface. Left atrium sits most...
   Components: 4
     #1: [The right ventricle]
     #2: [Left atrium sits]
     #3: [LV makes up]
     #4: [RV forms the]

📋 Scene 2: Left Atrial Enlargement Complications
   Layout: LAYOUT B - top_banner_three_below
   Narration: Mitral stenosis causes the left atrium to enlarge significantly. Posterior expan...
   Components: 4
     #1: [Mitral stenosis causes]
     #2: [Posterior expansion compresses]
     #3: [Recurrent laryngeal nerve]
```

```
      #4: [This specific clinical]

📋 Scene 3: Pericardium Layers and Innervation
   Layout: LAYOUT C - center_focus
   Narration: Three layers surround the heart, including the fibrous pericardium. The pericard...
   Components: 4
     #1: [Three layers surround]
     #2: [The pericardial space]
     #3: [Phrenic nerves provide]
     #4: [Pericarditis pain often]

📋 Scene 4: Coronary Supply and Dominance
   Layout: LAYOUT D - timeline_horizontal
   Narration: The LAD supplies the anterior septum and LV surface. PDA origin determines the h...
   Components: 4
     #1: [The LAD supplies]
     #2: [PDA origin determines]
     #3: [Most people have]
     #4: [Coronary flow to]

✅ Video plan ready: 4 scenes
```

Step 4: Generate Full Narration Audio

Concatenate all scene narrations into one continuous script, then generate audio with word timestamps.

```
# Build full narration with scene markers
full_narration = ""
scene_text_ranges = []  # track character positions per scene

for i, scene in enumerate(video_plan['scenes']):
    start_char = len(full_narration)

    # Add a brief pause between scenes (period + space)
    if i > 0:
        full_narration += " "

    full_narration += scene['narration'].strip()
    end_char = len(full_narration)
    scene_text_ranges.append((start_char, end_char))

print(f"📝 Full narration: {len(full_narration)} chars")
print(f"   Preview: {full_narration[:150]}...")
print(f"\n🎙 Generating audio...")

# ElevenLabs TTS
ELEVENLABS_URL = f"https://api.elevenlabs.io/v1/text-to-speech/{VOICE_ID}/with-timestamps"

tts_payload = {
    "text": full_narration,
    "model_id": "eleven_multilingual_v2",
    "voice_settings": {
        "stability": 0.5,
        "similarity_boost": 0.75,
        "style": 0.3
    }
}

tts_headers = {
    "Content-Type": "application/json",
    "xi-api-key": ELEVENLABS_API_KEY
}

tts_response = requests.post(ELEVENLABS_URL, json=tts_payload, headers=tts_headers)

if tts_response.status_code != 200:
    print(f"❌ ElevenLabs error {tts_response.status_code}: {tts_response.text[:500]}")
    raise Exception("TTS failed")

tts_data = tts_response.json()

# Save audio
audio_bytes = base64.b64decode(tts_data['audio_base64'])
AUDIO_FILE = "full_narration.mp3"
with open(AUDIO_FILE, 'wb') as f:
    f.write(audio_bytes)

print(f"✅ Audio saved: {AUDIO_FILE}")

# Extract word timestamps
word_timestamps = []
```

```
if 'alignment' in tts_data:
    chars = tts_data['alignment']['characters']
    char_starts = tts_data['alignment']['character_start_times_seconds']
    char_ends = tts_data['alignment']['character_end_times_seconds']

    current_word = ""
    word_start = 0
    for i, char in enumerate(chars):
        if char == ' ' or i == len(chars) - 1:
            if i == len(chars) - 1 and char != ' ':
                current_word += char
            if current_word.strip():
                word_timestamps.append({
                    'word': current_word.strip(),
                    'start': word_start,
                    'end': char_ends[i-1] if char == ' ' else char_ends[i]
                })
            current_word = ""
            if i + 1 < len(chars):
                word_start = char_starts[i+1]
        else:
            if current_word == "":
                word_start = char_starts[i]
            current_word += char
elif 'word_timestamps' in tts_data:
    word_timestamps = tts_data['word_timestamps']

total_audio_duration = word_timestamps[-1]['end'] if word_timestamps else 30
print(f"⏱ Duration: {total_audio_duration:.1f}s ({len(word_timestamps)} words)")

# Preview first 15 words
for wt in word_timestamps[:15]:
    print(f"  {wt['start']:.2f}s: \"{wt['word']}\"")

display(Audio(AUDIO_FILE))
```

```
📝 Full narration: 963 chars
   Preview: The right ventricle forms the most anterior heart surface. Left atrium sits most posterior near the e

🎙 Generating audio...
✅ Audio saved: full_narration.mp3
⏱ Duration: 67.5s (138 words)
  0.00s: "The"
  0.28s: "right"
  0.49s: "ventricle"
  1.02s: "forms"
  1.35s: "the"
  1.46s: "most"
  1.68s: "anterior"
  2.24s: "heart"
  2.50s: "surface."
  3.74s: "Left"
  4.05s: "atrium"
  4.55s: "sits"
  4.85s: "most"
  5.08s: "posterior"
  5.75s: "near"
```

0:00 / 1:07

Step 5: Build Global Sync Timeline

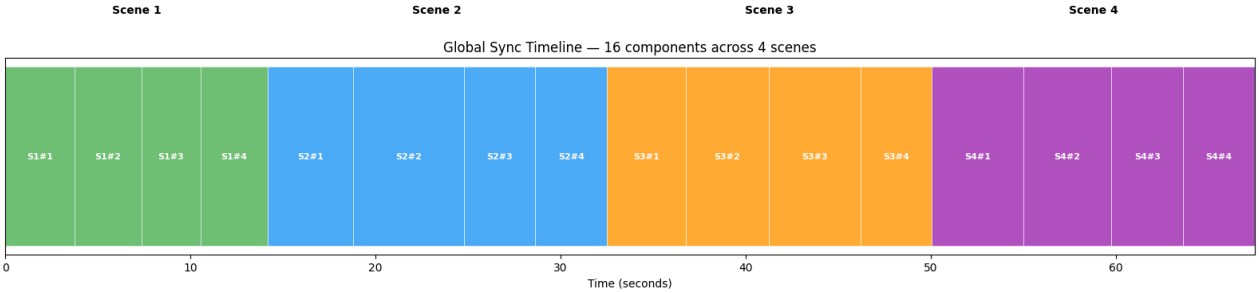
Maps every component across all scenes to exact timestamps in the audio.

```
def find_trigger_time(trigger_phrase, word_timestamps, search_after=0):
    """
    Find timestamp of trigger phrase, searching only AFTER a given time.
    This prevents matching a phrase in an earlier scene.
    """
    trigger_words = trigger_phrase.lower().split()
    words_lower = [w['word'].lower().strip('.,!?;:\'\"') for w in word_timestamps]

    for i in range(len(words_lower) - len(trigger_words) + 1):
        if word_timestamps[i]['start'] < search_after - 0.5:
            continue
        match = True
        for j, tw in enumerate(trigger_words):
            if tw not in words_lower[i + j]:
                match = False
                break
```

```
        if match:
            return word_timestamps[i]['start']

    # Fallback: find first matching word after search_after
    for tw in trigger_words:
        for i, w in enumerate(words_lower):
            if tw in w and word_timestamps[i]['start'] >= search_after - 0.5:
                return word_timestamps[i]['start']

    return None


# Build global timeline
print("⏱️ Building global sync timeline...\n")

global_timeline = []  # flat list of all components across all scenes
search_after = 0

for scene in video_plan['scenes']:
    scene_num = scene['scene_number']
    print(f"  Scene {scene_num}: {scene['scene_title']}")

    for sync_item in scene['sync_map']:
        trigger = sync_item['trigger_phrase']
        start_time = find_trigger_time(trigger, word_timestamps, search_after)

        if start_time is None:
            # Fallback: place evenly within estimated scene time
            scene_idx = scene_num - 1
            comp_idx = sync_item['component'] - 1
            n_total = NUM_SCENES * COMPONENTS_PER_SCENE
            global_idx = scene_idx * COMPONENTS_PER_SCENE + comp_idx
            start_time = (global_idx / n_total) * total_audio_duration
            print(f"    ⚠️ '{trigger}' not found, fallback: {start_time:.2f}s")
        else:
            print(f"    ✅ #{sync_item['component']} [{trigger}] → {start_time:.2f}s")

        global_timeline.append({
            'scene': scene_num,
            'component': sync_item['component'],
            'start_time': start_time,
            'trigger': trigger,
            'narration': sync_item['narration_segment']
        })

        search_after = start_time + 0.5

# Calculate end times
for i in range(len(global_timeline)):
    if i < len(global_timeline) - 1:
        global_timeline[i]['end_time'] = global_timeline[i + 1]['start_time']
    else:
        global_timeline[i]['end_time'] = total_audio_duration
    global_timeline[i]['duration'] = global_timeline[i]['end_time'] - global_timeline[i]['start_time']

# Mark scene transitions (where scene number changes)
for i in range(1, len(global_timeline)):
    if global_timeline[i]['scene'] != global_timeline[i-1]['scene']:
        global_timeline[i]['scene_transition'] = True
    else:
        global_timeline[i]['scene_transition'] = False
global_timeline[0]['scene_transition'] = True  # first scene starts

# Visual timeline
fig, ax = plt.subplots(figsize=(16, 4))
scene_colors = ['#4CAF50', '#2196F3', '#FF9800', '#9C27B0', '#F44336',
                '#00BCD4', '#FF5722', '#607D8B', '#E91E63', '#3F51B5']

for i, t in enumerate(global_timeline):
    color = scene_colors[(t['scene']-1) % len(scene_colors)]
    ax.barh(0, t['duration'], left=t['start_time'], height=0.5,
            color=color, alpha=0.8, edgecolor='white', linewidth=0.5)
    if t['duration'] > 2:
        ax.text(t['start_time'] + t['duration']/2, 0,
                f"S{t['scene']}#{t['component']}",
                ha='center', va='center', fontsize=8, color='white', fontweight='bold')

# Scene labels on top
for scene in video_plan['scenes']:
    scene_items = [t for t in global_timeline if t['scene'] == scene['scene_number']]
    if scene_items:
        s_start = scene_items[0]['start_time']
        s_end = scene_items[-1]['end_time']
```

```
        ax.text((s_start + s_end)/2, 0.4, f"Scene {scene['scene_number']}",
                ha='center', fontsize=10, fontweight='bold')

ax.set_xlim(0, total_audio_duration)
ax.set_xlabel('Time (seconds)')
ax.set_yticks([])
ax.set_title(f'Global Sync Timeline — {len(global_timeline)} components across {NUM_SCENES} scenes')
plt.tight_layout()
plt.show()

print(f"\n✅ Timeline: {len(global_timeline)} draw events over {total_audio_duration:.1f}s")
```

```
⏱️ Building global sync timeline...

  Scene 1: Heart Orientation and Surfaces
    ✅ #1 [The right ventricle] → 0.00s
    ✅ #2 [Left atrium sits] → 3.74s
    ✅ #3 [LV makes up] → 7.34s
    ✅ #4 [RV forms the] → 10.53s
  Scene 2: Left Atrial Enlargement Complications
    ✅ #1 [Mitral stenosis causes] → 14.15s
    ✅ #2 [Posterior expansion compresses] → 18.80s
    ✅ #3 [Recurrent laryngeal nerve] → 24.76s
    ✅ #4 [This specific clinical] → 28.64s
  Scene 3: Pericardium Layers and Innervation
    ✅ #1 [Three layers surround] → 32.51s
    ✅ #2 [The pericardial space] → 36.76s
    ✅ #3 [Phrenic nerves provide] → 41.25s
    ✅ #4 [Pericarditis pain often] → 46.20s
  Scene 4: Coronary Supply and Dominance
    ✅ #1 [The LAD supplies] → 50.07s
    ✅ #2 [PDA origin determines] → 55.01s
    ✅ #3 [Most people have] → 59.78s
    ✅ #4 [Coronary flow to] → 63.65s
```




```
✅ Timeline: 16 draw events over 67.5s
```

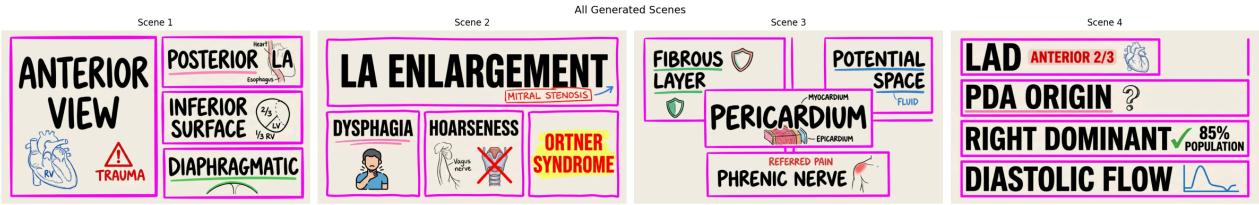
Step 6: Generate All Scene Frames

Generates one whiteboard image per scene using Gemini Image.

```
os.makedirs("scenes", exist_ok=True)

scene_frames = []  # stores frame data per scene

for scene in video_plan['scenes']:
    scene_num = scene['scene_number']
    print(f"\n🎨 Generating Scene {scene_num}/{NUM_SCENES}: {scene['scene_title']}")

    # Rate limiting — Gemini image gen can be slow
    if scene_num > 1:
        print("   Waiting 5s (rate limit)...")
        time.sleep(5)

    try:
        img_response = gemini_client.models.generate_content(
            model="gemini-3.1-flash-image-preview",
            contents=scene['image_prompt'],
            config=types.GenerateContentConfig(
                response_modalities=["IMAGE"],
                image_config=types.ImageConfig(
                    aspect_ratio="16:9"
```

```
                    aspect_ratio="16:9",
                )
            )
        )

        frame_file = f"scenes/scene_{scene_num}.png"
        for part in img_response.parts:
            if part.inline_data is not None:
                with open(frame_file, 'wb') as f:
                    f.write(part.inline_data.data)
                break

        frame_img = cv2.imread(frame_file)
        frame_rgb = cv2.cvtColor(frame_img, cv2.COLOR_BGR2RGB)

        scene_frames.append({
            'scene_number': scene_num,
            'file': frame_file,
            'image_bgr': frame_img,
            'image_rgb': frame_rgb,
            'height': frame_img.shape[0],
            'width': frame_img.shape[1]
        })
        print(f"   ✅ Generated: {frame_img.shape[1]}x{frame_img.shape[0]}")

    except Exception as e:
        print(f"   ❌ Error: {e}")
        print(f"   Retrying in 10s...")
        time.sleep(10)
        # Retry once
        try:
            img_response = gemini_client.models.generate_content(
                model="gemini-2.5-flash-preview-image-generation",
                contents=scene['image_prompt'],
                config=types.GenerateContentConfig(
                    response_modalities=["IMAGE"],
                    image_config=types.ImageConfig(
                        aspect_ratio="16:9",
                    )
                )
            )
            frame_file = f"scenes/scene_{scene_num}.png"
            for part in img_response.parts:
                if part.inline_data is not None:
                    with open(frame_file, 'wb') as f:
                        f.write(part.inline_data.data)
                    break
            frame_img = cv2.imread(frame_file)
            frame_rgb = cv2.cvtColor(frame_img, cv2.COLOR_BGR2RGB)
            scene_frames.append({
                'scene_number': scene_num,
                'file': frame_file,
                'image_bgr': frame_img,
                'image_rgb': frame_rgb,
                'height': frame_img.shape[0],
                'width': frame_img.shape[1]
            })
            print(f"   ✅ Retry succeeded")
        except Exception as e2:
            print(f"   ❌ Retry failed: {e2}")

# Preview all scenes
n_scenes = len(scene_frames)
fig, axes = plt.subplots(1, n_scenes, figsize=(6*n_scenes, 4))
if n_scenes == 1:
    axes = [axes]
for i, sf in enumerate(scene_frames):
    axes[i].imshow(sf['image_rgb'])
    axes[i].set_title(f"Scene {sf['scene_number']}")
    axes[i].axis('off')
plt.suptitle("All Generated Scenes", fontsize=14)
plt.tight_layout()
plt.show()

print(f"\n✅ {n_scenes} scene frames generated")
```

```
🎨 Generating Scene 1/4: Heart Orientation and Surfaces
   ✅ Generated: 1376x768

🎨 Generating Scene 2/4: Left Atrial Enlargement Complications
   Waiting 5s (rate limit)...
   ✅ Generated: 1376x768

🎨 Generating Scene 3/4: Pericardium Layers and Innervation
   Waiting 5s (rate limit)...
   ✅ Generated: 1376x768

🎨 Generating Scene 4/4: Coronary Supply and Dominance
   Waiting 5s (rate limit)...
   ✅ Generated: 1376x768
```




```
✅ 4 scene frames generated
```

Step 7: Extract Components from ALL Scenes

```
padding = 12

def extract_components_from_frame(frame_bgr, scene_number):
    """Extract magenta-bordered components with aggressive magenta removal.
       Handles cream/off-white backgrounds."""
    h, w = frame_bgr.shape[:2]
    hsv = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2HSV)

    # Detect the background color (sample from corners, avoiding borders)
    corners = [
        frame_bgr[5:15, 5:15],           # top-left
        frame_bgr[5:15, w-15:w-5],       # top-right
        frame_bgr[h-15:h-5, 5:15],       # bottom-left
        frame_bgr[h-15:h-5, w-15:w-5],   # bottom-right
    ]
    bg_color = np.median(np.concatenate([c.reshape(-1, 3) for c in corners], axis=0), axis=0).astype(np.uint8)
    print(f"    Background color detected: BGR({bg_color[0]}, {bg_color[1]}, {bg_color[2]})")

    # Detection mask for finding borders
    mag_detect = cv2.inRange(hsv, (140, 100, 100), (170, 255, 255))
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    mag_dilated = cv2.dilate(mag_detect, kernel, iterations=1)

    contours, _ = cv2.findContours(mag_dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    min_area = (w * h) * 0.03
    bboxes = []
    for cnt in contours:
        x, y, bw, bh = cv2.boundingRect(cnt)
        if bw * bh > min_area:
            bboxes.append((x, y, bw, bh))

    bboxes.sort(key=lambda b: (b[1] // (h // 3), b[0]))

    # Trim excess components
    if len(bboxes) > COMPONENTS_PER_SCENE:
        print(f"    ⚠️ Found {len(bboxes)} boxes, trimming to {COMPONENTS_PER_SCENE} largest")
        bboxes.sort(key=lambda b: b[2] * b[3], reverse=True)
        bboxes = bboxes[:COMPONENTS_PER_SCENE]
        bboxes.sort(key=lambda b: (b[1] // (h // 3), b[0]))

    components = []
    for i, (x, y, bw, bh) in enumerate(bboxes):
        x1 = min(x + padding, w)
        y1 = min(y + padding, h)
        x2 = min(x + bw - padding, w)
```

```
        x2 = min(x + bw - padding, w)
        y2 = min(y + bh - padding, h)

        if x2 <= x1 or y2 <= y1:
            continue

        crop = frame_bgr[y1:y2, x1:x2].copy()

        # === AGGRESSIVE MAGENTA REMOVAL ===
        crop_hsv = cv2.cvtColor(crop, cv2.COLOR_BGR2HSV)

        mag1 = cv2.inRange(crop_hsv, (135, 50, 50), (175, 255, 255))
        mag2 = cv2.inRange(crop_hsv, (140, 30, 100), (170, 255, 255))
        mag3 = cv2.inRange(crop_hsv, (145, 80, 150), (165, 255, 255))
        crop_mag_mask = cv2.bitwise_or(mag1, cv2.bitwise_or(mag2, mag3))

        b_ch, g_ch, r_ch = cv2.split(crop)
        rgb_mag = ((r_ch.astype(int) > 150) &
                   (g_ch.astype(int) < 100) &
                   (b_ch.astype(int) > 150)).astype(np.uint8) * 255
        crop_mag_mask = cv2.bitwise_or(crop_mag_mask, rgb_mag)

        dilate_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (7, 7))
        crop_mag_mask = cv2.dilate(crop_mag_mask, dilate_kernel, iterations=2)

        edge_w = 5
        crop_mag_mask[:edge_w, :] = 255
        crop_mag_mask[-edge_w:, :] = 255
        crop_mag_mask[:, :edge_w] = 255
        crop_mag_mask[:, -edge_w:] = 255

        # Inpaint pass 1
        clean = cv2.inpaint(crop, crop_mag_mask, 7, cv2.INPAINT_TELEA)

        # Inpaint pass 2
        clean_hsv = cv2.cvtColor(clean, cv2.COLOR_BGR2HSV)
        remaining = cv2.inRange(clean_hsv, (135, 40, 50), (175, 255, 255))
        if cv2.countNonZero(remaining) > 0:
            remaining = cv2.dilate(remaining,
                                   cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5)),
                                   iterations=1)
            clean = cv2.inpaint(clean, remaining, 5, cv2.INPAINT_TELEA)

        # Force edges to background color (cream, not white)
        clean[:3, :] = bg_color
        clean[-3:, :] = bg_color
        clean[:, :3] = bg_color
        clean[:, -3:] = bg_color

        clean_rgb = cv2.cvtColor(clean, cv2.COLOR_BGR2RGB)

        components.append({
            'id': i + 1,
            'scene': scene_number,
            'image': clean_rgb,
            'position': {'x': x, 'y': y},
            'size': {'width': bw, 'height': bh}
        })

    return components


# Process all scenes
all_scene_components = {}

print("Extracting components from all scenes...\n")
for sf in scene_frames:
    sn = sf['scene_number']
    comps = extract_components_from_frame(sf['image_bgr'], sn)
    all_scene_components[sn] = comps
    print(f"  Scene {sn}: {len(comps)} components extracted")

    if len(comps) == 0:
        print(f"    ⚠️ No magenta borders detected! May need manual boxes.")

# Preview
for sn, comps in all_scene_components.items():
    if comps:
        fig, axes = plt.subplots(1, len(comps), figsize=(4*len(comps), 3))
        if len(comps) == 1:
            axes = [axes]
        for i, c in enumerate(comps):
            axes[i].imshow(c['image'])
            axes[i].set_title(f"C{sn}#{c['id']}")
```

```
            axes[i].set_title(f"S{sn}#{c['id']}")
            axes[i].axis('off')
        plt.suptitle(f"Scene {sn} Components", fontsize=12)
        plt.tight_layout()
        plt.show()

print(f"\n✅ All components extracted")
```

```
Extracting components from all scenes...

    Background color detected: BGR(227, 239, 243)
  Scene 1: 4 components extracted
    Background color detected: BGR(227, 239, 243)
  Scene 2: 4 components extracted
    Background color detected: BGR(228, 239, 243)
  Scene 3: 1 components extracted
    Background color detected: BGR(227, 239, 245)
  Scene 4: 4 components extracted
```

Scene 1 Components

S1#1

S1#2

S1#3

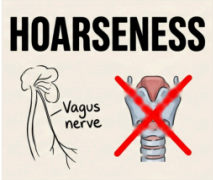
S1#4

Scene 2 Components

S2#1

S2#2

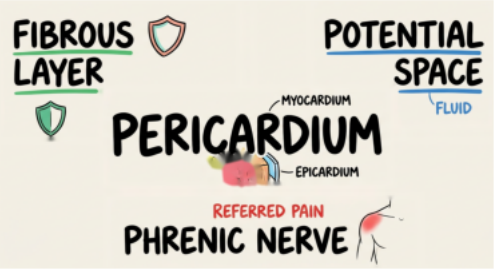
S2#3



S2#4

Scene 3 Components

S3#1



Scene 4 Components

S4#1

S4#2

S4#3

S4#4

```
✅ All components extracted
```

Step 8: Build Stroke Paths for All Components

```
import io as _io
```

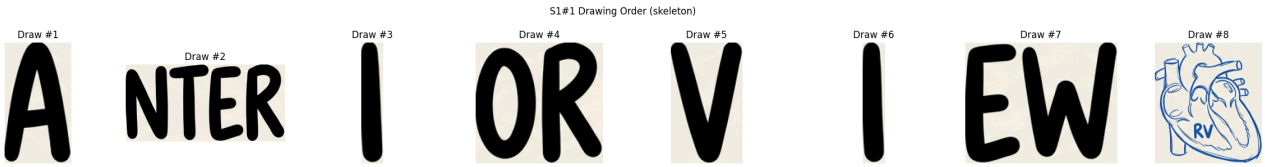
```
import io as _io
import re

def extract_ink_mask(img, threshold=200):
    """Extract ink mask with lower threshold for cream backgrounds."""
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    ink = gray < threshold
    ink = cv2.morphologyEx(ink.astype(np.uint8), cv2.MORPH_CLOSE, np.ones((2,2), np.uint8)).astype(bool)
    return ink


def find_connected_elements(ink_mask, min_pixels=30):
    """Find individual visual elements using connected component analysis."""
    h, w = ink_mask.shape

    connect_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 3))
    connected = cv2.dilate(ink_mask.astype(np.uint8), connect_kernel, iterations=2)

    num_labels, labels = cv2.connectedComponents(connected)

    elements = []
    for label_id in range(1, num_labels):
        component_region = (labels == label_id)
        original_ink = ink_mask & component_region
        area = np.sum(original_ink)

        if area < min_pixels:
            continue

        rows, cols = np.where(original_ink)
        if len(rows) == 0:
            continue

        y1, y2 = rows.min(), rows.max()
        x1, x2 = cols.min(), cols.max()

        elements.append({
            'mask': original_ink,
            'bbox': (x1, y1, x2 - x1 + 1, y2 - y1 + 1),
            'center': (int(np.mean(rows)), int(np.mean(cols))),
            'area': int(area),
            'y_min': int(y1),
            'x_min': int(x1),
        })

    return elements


def order_elements_reading(elements):
    """Order elements top-to-bottom, left-to-right within rows."""
    if len(elements) == 0:
        return elements

    elements_sorted = sorted(elements, key=lambda e: e['center'][0])

    rows = []
    current_row = [elements_sorted[0]]

    for i in range(1, len(elements_sorted)):
        prev_bbox = current_row[-1]['bbox']
        curr_bbox = elements_sorted[i]['bbox']
        prev_y_range = (current_row[-1]['y_min'], current_row[-1]['y_min'] + prev_bbox[3])
        curr_y_range = (elements_sorted[i]['y_min'], elements_sorted[i]['y_min'] + curr_bbox[3])

        overlap_start = max(prev_y_range[0], curr_y_range[0])
        overlap_end = min(prev_y_range[1], curr_y_range[1])
        overlap = max(0, overlap_end - overlap_start)
        min_height = min(prev_bbox[3], curr_bbox[3])

        if min_height > 0 and overlap / min_height > 0.4:
            current_row.append(elements_sorted[i])
        else:
            rows.append(current_row)
            current_row = [elements_sorted[i]]

    rows.append(current_row)

    ordered = []
    for row in rows:
        row_sorted = sorted(row, key=lambda e: e['x_min'])
        ordered.extend(row_sorted)

    return ordered
```

```
def order_elements_with_gemini(comp_image, elements, gemini_client):
    """Use Gemini Vision to determine drawing order. Fixed prompt."""
    if len(elements) <= 2:
        return order_elements_reading(elements)

    try:
        annotated = comp_image.copy()
        for i, elem in enumerate(elements):
            x, y, bw, bh = elem['bbox']
            cv2.rectangle(annotated, (x, y), (x + bw, y + bh), (255, 0, 0), 2)
            # Larger font, white background for readability
            label = str(i + 1)
            (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.8, 2)
            cv2.rectangle(annotated, (x, y - th - 6), (x + tw + 4, y), (255, 255, 255), -1)
            cv2.putText(annotated, label, (x + 2, y - 4),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 0), 2)

        annotated_bgr = cv2.cvtColor(annotated, cv2.COLOR_RGB2BGR)
        _, img_bytes = cv2.imencode('.png', annotated_bgr)

        n = len(elements)
        prompt = f"""Look at this whiteboard image. There are exactly {n} visual elements, each marked with a BLU

I need you to tell me the order a human would DRAW these elements on a whiteboard.

Rules:
- Large title text is drawn FIRST
- Then supporting illustrations/icons
- Then smaller labels/subtitles LAST
- Top elements before bottom elements
- Left elements before right elements

You MUST respond with EXACTLY {n} numbers, comma-separated, no spaces, no trailing comma.
Every number from 1 to {n} must appear exactly once.

For example if there are 5 elements: 2,1,4,3,5

Your answer ({n} numbers):"""

        response = gemini_client.models.generate_content(
            model="gemini-2.5-flash",
            contents=[
                {
                    "parts": [
                        {"inline_data": {"mime_type": "image/png", "data": base64.b64encode(img_bytes.tobytes()).
                        {"text": prompt}
                    ]
                }
            ],
            config=types.GenerateContentConfig(
                temperature=0.1,
                max_output_tokens=200,
            )
        )

        order_text = response.text.strip()
        numbers = [int(n_val) for n_val in re.findall(r'\d+', order_text)]

        valid_range = set(range(1, n + 1))
        if set(numbers) == valid_range and len(numbers) == n:
            reordered = [elements[num - 1] for num in numbers]
            print(f"      ✅ Gemini order: {numbers}")
            return reordered
        else:
            print(f"      ⚠️ Gemini returned: {order_text} (expected {n} numbers), using reading order")
            return order_elements_reading(elements)

    except Exception as e:
        print(f"      ⚠️ Gemini failed: {e}, using reading order")
        return order_elements_reading(elements)


def build_element_stroke_path(element_mask, num_anchors=150):
    """Build a stroke path for a SINGLE element using skeleton + nearest-neighbor.
    Works well because each element is small (one letter, one icon)."""
    h, w = element_mask.shape
    rows, cols = np.where(element_mask)
    if len(rows) == 0:
        return np.array([[h // 2, w // 2]])

    skeleton = skeletonize(element_mask)
```

```
    skel_points = np.array(np.where(skeleton)).T

    if len(skel_points) < 5:
        # Fallback: use ink points directly
        ink_points = np.column_stack((rows, cols))
        step = max(1, len(ink_points) // num_anchors)
        sorted_idx = np.lexsort((ink_points[:, 1], ink_points[:, 0]))
        return ink_points[sorted_idx[::step]]

    if len(skel_points) > num_anchors:
        indices = np.linspace(0, len(skel_points) - 1, num_anchors, dtype=int)
        pts = skel_points[indices]
    else:
        pts = skel_points

    # Start from top-left-most point
    start_scores = pts[:, 1] * 1.0 + pts[:, 0] * 0.3
    start_idx = int(np.argmin(start_scores))

    remaining = list(range(len(pts)))
    current = start_idx
    ordered = [current]
    remaining.remove(current)

    if len(pts) > 1:
        tree = KDTree(pts)
        while remaining:
            _, indices = tree.query(pts[current], k=len(pts))
            for idx in indices:
                if idx in remaining:
                    current = idx
                    ordered.append(current)
                    remaining.remove(current)
                    break

    return pts[ordered]


def build_element_distance_map(element_mask):
    """Build distance transform from element boundary inward.
    Used for outline_fill reveal style."""
    mask_uint8 = element_mask.astype(np.uint8) * 255

    # Distance from the boundary of the element
    dist = cv2.distanceTransform(mask_uint8, cv2.DIST_L2, 5)

    # Normalize to 0-1 range
    max_dist = dist.max()
    if max_dist > 0:
        dist_norm = dist / max_dist
    else:
        dist_norm = dist

    return dist_norm


def build_element_expand_map(element_mask):
    """Build distance from center of element outward.
    Used for expand reveal style."""
    rows, cols = np.where(element_mask)
    if len(rows) == 0:
        return np.zeros_like(element_mask, dtype=np.float32)

    cy, cx = int(np.mean(rows)), int(np.mean(cols))
    h, w = element_mask.shape

    yy, xx = np.mgrid[0:h, 0:w]
    dist = np.sqrt((yy - cy)**2 + (xx - cx)**2).astype(np.float32)

    # Only keep distances for ink pixels
    dist_masked = np.where(element_mask, dist, 0)

    max_dist = dist_masked.max()
    if max_dist > 0:
        dist_norm = dist_masked / max_dist
    else:
        dist_norm = dist_masked

    return dist_norm


# ---- Build all data for all components ----
```

```
USE_GEMINI_ORDERING = True
REVEAL_STYLE = 'skeleton'  # ← CHANGE THIS: 'skeleton', 'outline_fill', 'expand'

all_stroke_data = {}

print(f"Building element maps (reveal style: {REVEAL_STYLE})...\n")
for sn, comps in all_scene_components.items():
    all_stroke_data[sn] = []
    for comp in comps:
        ink_mask = extract_ink_mask(comp['image'])
        elements = find_connected_elements(ink_mask)

        print(f"  S{sn}#{comp['id']}: {np.sum(ink_mask)} ink px, {len(elements)} elements")

        # Order elements
        if USE_GEMINI_ORDERING and len(elements) > 2:
            ordered = order_elements_with_gemini(comp['image'], elements, gemini_client)
        else:
            ordered = order_elements_reading(elements)

        # Pre-compute reveal data per element based on style
        for elem in ordered:
            if REVEAL_STYLE == 'skeleton':
                elem['stroke_path'] = build_element_stroke_path(elem['mask'])
            elif REVEAL_STYLE == 'outline_fill':
                elem['dist_map'] = build_element_distance_map(elem['mask'])
            elif REVEAL_STYLE == 'expand':
                elem['expand_map'] = build_element_expand_map(elem['mask'])

        all_stroke_data[sn].append({
            'ink_mask': ink_mask,
            'elements': ordered,
            'num_elements': len(ordered),
        })

print(f"\n✅ All element maps built (style: {REVEAL_STYLE})")

# Preview element order for first component
if all_stroke_data and 1 in all_stroke_data and len(all_stroke_data[1]) > 0:
    sd = all_stroke_data[1][0]
    comp_img = all_scene_components[1][0]
    n_elems = min(sd['num_elements'], 8)

    if n_elems > 0:
        fig, axes = plt.subplots(1, n_elems, figsize=(3 * n_elems, 3))
        if n_elems == 1:
            axes = [axes]
        for i in range(n_elems):
            elem = sd['elements'][i]
            x, y, bw, bh = elem['bbox']
            crop = comp_img['image'][y:y+bh, x:x+bw]
            axes[i].imshow(crop)
            axes[i].set_title(f"Draw #{i+1}")
            axes[i].axis('off')
        plt.suptitle(f"S1#1 Drawing Order ({REVEAL_STYLE})", fontsize=12)
        plt.tight_layout()
        plt.show()
```

```
Building element maps (reveal style: skeleton)...

  S1#1: 89749 ink px, 10 elements
      ⚠️ Gemini returned: 3,1,4,2 (expected 10 numbers), using reading order
  S1#2: 28055 ink px, 2 elements
  S1#3: 35738 ink px, 5 elements
      ⚠️ Gemini returned: 1,2,3 (expected 5 numbers), using reading order
  S1#4: 34485 ink px, 3 elements
      ✅ Gemini order: [1, 2, 3]
  S2#1: 116268 ink px, 11 elements
      ⚠️ Gemini returned: 2,3,4, (expected 11 numbers), using reading order
  S2#2: 29137 ink px, 2 elements
  S2#3: 35812 ink px, 3 elements
      ✅ Gemini order: [1, 2, 3]
  S2#4: 29872 ink px, 2 elements
  S3#1: 155491 ink px, 15 elements
      ⚠️ Gemini returned: 8,2,3, (expected 15 numbers), using reading order
  S4#1: 52836 ink px, 8 elements
      ⚠️ Gemini returned: 3,4,5 (expected 8 numbers), using reading order
  S4#2: 33899 ink px, 4 elements
      ✅ Gemini order: [2, 1, 3, 4]
  S4#3: 70864 ink px, 11 elements
      ⚠️ Gemini returned: 2,3,4,5, (expected 11 numbers), using reading order
  S4#4: 62219 ink px, 7 elements
      ⚠️ Gemini returned: 5,2,6,3, (expected 7 numbers), using reading order

✅ All element maps built (style: skeleton)
```



Step 9: Pen Cursor

```
def create_pen_cursor(size=60):
    cursor = np.zeros((size, size, 4), dtype=np.uint8)
    img = Image.fromarray(cursor)
    draw = ImageDraw.Draw(img)
    draw.polygon([(3,3),(15,20),(size-5,size-5),(25,15)], fill=(50,50,50,220))
    draw.polygon([(3,3),(12,18),(18,12)], fill=(20,20,20,255))
    draw.line([(20,18),(size-15,size-15)], fill=(80,80,80,200), width=2)
    return np.array(img)

def overlay_cursor(canvas, cursor, x, y):
    ch, cw = cursor.shape[:2]
    H, W = canvas.shape[:2]
    x1, y1 = x-3, y-3
    x2, y2 = x1+cw, y1+ch
    sx, sy = max(0,-x1), max(0,-y1)
    x1, y1 = max(0,x1), max(0,y1)
    x2, y2 = min(W,x2), min(H,y2)
    if x2<=x1 or y2<=y1: return canvas
    rh, rw = y2-y1, x2-x1
    cr = cursor[sy:sy+rh, sx:sx+rw]
    a = cr[:,:,3:4].astype(np.float32)/255
    canvas[y1:y2,x1:x2] = (cr[:,:,:3].astype(np.float32)*a + canvas[y1:y2,x1:x2].astype(np.float32)*(1-a)).astyp
    return canvas

pen_cursor = create_pen_cursor(60)
print("✅ Pen cursor ready")
```

```
✅ Pen cursor ready
```

Step 10: Render Full Multi-Scene Video 🎬

The big render — handles scene transitions, per-component sync, stroke animation.

```
_corners = [
    scene_frames[0]['image_bgr'][5:15, 5:15],
    scene_frames[0]['image_bgr'][5:15, -15:-5],
```

```
        scene_frames[0]['image_bgr'][-15:-5, 5:15],
        scene_frames[0]['image_bgr'][-15:-5, -15:-5],
    ]
    BG_COLOR_BGR = np.median(np.concatenate([c.reshape(-1, 3) for c in _corners], axis=0), axis=0).astype(np.uint8)
    BG_COLOR_RGB = BG_COLOR_BGR[::-1]
    print(f"Background color: RGB({BG_COLOR_RGB[0]}, {BG_COLOR_RGB[1]}, {BG_COLOR_RGB[2]})")

    FRAME_W = scene_frames[0]['width']
    FRAME_H = scene_frames[0]['height']

    VIDEO_ONLY = 'video_only.mp4'
    FINAL_OUTPUT = 'whiteboard_video_final.mp4'

    total_frames = int((total_audio_duration + 2) * FPS)
    transition_frames = int(TRANSITION_DURATION * FPS)

    print(f"🎬 Rendering multi-scene video (reveal: {REVEAL_STYLE})")
    print(f"   Canvas: {FRAME_W}x{FRAME_H}")
    print(f"   Frames: {total_frames} ({total_audio_duration+2:.1f}s)")
    print(f"   Scenes: {NUM_SCENES}")
    print(f"   Transition: {TRANSITION_STYLE} ({TRANSITION_DURATION}s)")
    print()

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(VIDEO_ONLY, fourcc, FPS, (FRAME_W, FRAME_H))

    current_scene = None
    canvas = np.full((FRAME_H, FRAME_W, 3), BG_COLOR_RGB, dtype=np.uint8)
    scene_reveal_masks = {}
    scene_committed = {}
    prev_cx, prev_cy = 0, 0
    in_transition = False
    transition_start_frame = 0
    pre_transition_canvas = None

    entry_frame_ranges = []
    for t in global_timeline:
        sf = int(t['start_time'] * FPS)
        ef = int(t['end_time'] * FPS)
        entry_frame_ranges.append((sf, ef))

    print("Rendering...")
    render_start = time.time()

    for frame_num in range(total_frames):
        active_scene = None
        for t in global_timeline:
            if t['start_time'] <= frame_num / FPS <= t['end_time']:
                active_scene = t['scene']
                break
        if active_scene is None:
            if global_timeline:
                active_scene = global_timeline[-1]['scene']
            else:
                active_scene = 1

        # ---- SCENE TRANSITION ----
        if active_scene != current_scene:
            if current_scene is not None:
                in_transition = True
                transition_start_frame = frame_num
                pre_transition_canvas = canvas.copy()
            current_scene = active_scene
            comps = all_scene_components.get(current_scene, [])
            scene_reveal_masks = {}
            scene_committed = {}
            for i, c in enumerate(comps):
                ch, cw = c['image'].shape[:2]
                scene_reveal_masks[i] = np.zeros((ch, cw), dtype=bool)
                scene_committed[i] = False

        # ---- TRANSITION EFFECT ----
        if in_transition:
            t_progress = (frame_num - transition_start_frame) / max(1, transition_frames)
            if t_progress >= 1.0:
                in_transition = False
                canvas = np.full((FRAME_H, FRAME_W, 3), BG_COLOR_RGB, dtype=np.uint8)
            else:
                if TRANSITION_STYLE == 'erase':
                    erase_x = int((1 - t_progress) * FRAME_W)
                    canvas = pre_transition_canvas.copy()
                    canvas[:, erase_x:] = BG_COLOR_RGB
                elif TRANSITION_STYLE == 'fade':
```

```
                alpha = t_progress
                bg_canvas = np.full_like(pre_transition_canvas, BG_COLOR_RGB)
                canvas = cv2.addWeighted(pre_transition_canvas, 1-alpha, bg_canvas, alpha, 0)
            else:
                canvas = np.full((FRAME_H, FRAME_W, 3), BG_COLOR_RGB, dtype=np.uint8)
                in_transition = False
            out.write(cv2.cvtColor(canvas, cv2.COLOR_RGB2BGR))
            if frame_num % (FPS * 3) == 0:
                elapsed = time.time() - render_start
                pct = frame_num / total_frames * 100
                print(f"  {pct:.0f}% ({frame_num}/{total_frames}) [{elapsed:.0f}s]", flush=True)
            continue

        # ---- DRAW COMPONENTS ----
        frame_canvas = canvas.copy()
        cursor_pos = None

        comps = all_scene_components.get(current_scene, [])
        strokes = all_stroke_data.get(current_scene, [])

        for entry_idx, t_entry in enumerate(global_timeline):
            if t_entry['scene'] != current_scene:
                continue

            comp_idx = t_entry['component'] - 1
            if comp_idx >= len(comps):
                continue

            comp = comps[comp_idx]
            start_f, end_f = entry_frame_ranges[entry_idx]

            cx = comp['position']['x'] + padding
            cy = comp['position']['y'] + padding
            ch, cw = comp['image'].shape[:2]
            cw = min(cw, FRAME_W - cx)
            ch = min(ch, FRAME_H - cy)

            if frame_num < start_f:
                continue

            elif scene_committed.get(comp_idx, False) or frame_num >= end_f:
                frame_canvas[cy:cy+ch, cx:cx+cw] = comp['image'][:ch, :cw]
                canvas[cy:cy+ch, cx:cx+cw] = comp['image'][:ch, :cw]
                scene_committed[comp_idx] = True

            else:
                # ---- ELEMENT-BY-ELEMENT REVEAL ----
                if comp_idx >= len(strokes):
                    continue

                sd = strokes[comp_idx]
                ink_mask = sd['ink_mask']
                elements = sd['elements']
                num_elements = sd['num_elements']

                if num_elements == 0:
                    continue

                progress = (frame_num - start_f) / max(1, end_f - start_f)
                elem_progress = progress * num_elements
                current_elem_idx = min(int(elem_progress), num_elements - 1)
                within_elem_progress = elem_progress - int(elem_progress)
                if current_elem_idx >= num_elements:
                    current_elem_idx = num_elements - 1
                    within_elem_progress = 1.0

                reveal = scene_reveal_masks[comp_idx]

                # Fully reveal all PREVIOUS elements
                for ei in range(current_elem_idx):
                    reveal |= elements[ei]['mask'][:ch, :cw]

                # Current element — reveal based on style
                if current_elem_idx < num_elements:
                    elem = elements[current_elem_idx]
                    elem_mask = elem['mask'][:ch, :cw]
                    ex, ey, ebw, ebh = elem['bbox']

                    if REVEAL_STYLE == 'skeleton':
                        # ---- STYLE A: Stroke path traces the shape ----
                        path = elem.get('stroke_path', None)
                        if path is not None and len(path) > 0:
                            path_idx = min(len(path) - 1, int(within_elem_progress * len(path)))
```

```
                    path_idx = min(len(path) - 1, int(within_elem_progress * len(path)))
                    for pi in range(path_idx + 1):
                        pr, pc = path[pi]
                        pr, pc = min(int(pr), ch - 1), min(int(pc), cw - 1)
                        r1 = max(0, pr - REVEAL_RADIUS)
                        r2 = min(ch, pr + REVEAL_RADIUS)
                        c1 = max(0, pc - REVEAL_RADIUS)
                        c2 = min(cw, pc + REVEAL_RADIUS)
                        if r2 <= r1 or c2 <= c1:
                            continue
                        rr, cc = np.meshgrid(np.arange(r1, r2), np.arange(c1, c2), indexing='ij')
                        dist = np.sqrt((rr - pr)**2 + (cc - pc)**2)
                        within = dist <= REVEAL_RADIUS
                        ink_region = elem_mask[r1:r2, c1:c2]
                        reveal[r1:r2, c1:c2] |= (within & ink_region)

                    # Cursor follows path
                    pen_r, pen_c = path[path_idx]
                    cursor_pos = (cx + int(min(pen_c, cw-1)), cy + int(min(pen_r, ch-1)))

            elif REVEAL_STYLE == 'outline_fill':
                # ---- STYLE B: Outline first, then fill inward ----
                dist_map = elem.get('dist_map', None)
                if dist_map is not None:
                    # progress 0→0.5: reveal outline (dist < threshold)
                    # progress 0.5→1: fill inward (threshold increases)
                    if within_elem_progress < 0.5:
                        # Outline phase: reveal pixels with small distance from edge
                        outline_progress = within_elem_progress / 0.5
                        threshold = outline_progress * 0.3  # only edge pixels
                    else:
                        # Fill phase: reveal more and more interior
                        fill_progress = (within_elem_progress - 0.5) / 0.5
                        threshold = 0.3 + fill_progress * 0.7

                    new_pixels = elem_mask & (dist_map[:ch, :cw] <= threshold)
                    reveal |= new_pixels

                    # Cursor at the leading edge
                    newly_revealed = new_pixels & ~reveal
                    new_rows, new_cols = np.where(elem_mask & (dist_map[:ch, :cw] > (threshold - 0.05)) & (di
                    if len(new_rows) > 0:
                        cursor_pos = (cx + int(np.mean(new_cols)), cy + int(np.mean(new_rows)))
                    else:
                        cursor_pos = (cx + elem['center'][1], cy + elem['center'][0])

            elif REVEAL_STYLE == 'expand':
                # ---- STYLE C: Circular expansion from center ----
                expand_map = elem.get('expand_map', None)
                if expand_map is not None:
                    # Reveal pixels closer to center first
                    eased = within_elem_progress ** 0.7  # faster start
                    new_pixels = elem_mask & (expand_map[:ch, :cw] <= eased) & (expand_map[:ch, :cw] > 0)
                    reveal |= new_pixels

                    # Cursor at expanding edge
                    edge_rows, edge_cols = np.where(elem_mask & (expand_map[:ch, :cw] > (eased - 0.05)) & (ex
                    if len(edge_rows) > 0:
                        cursor_pos = (cx + int(np.mean(edge_cols)), cy + int(np.mean(edge_rows)))
                    else:
                        cursor_pos = (cx + elem['center'][1], cy + elem['center'][0])

        # Apply reveal mask
        mask_3d = np.stack([reveal[:ch, :cw]]*3, axis=-1)
        region = frame_canvas[cy:cy+ch, cx:cx+cw].copy()
        region[mask_3d] = comp['image'][:ch, :cw][mask_3d]
        frame_canvas[cy:cy+ch, cx:cx+cw] = region

    # Pen cursor
    if SHOW_CURSOR and cursor_pos:
        cur_x, cur_y = cursor_pos
        if prev_cx > 0:
            cur_x = int(0.4*cur_x + 0.6*prev_cx)
            cur_y = int(0.4*cur_y + 0.6*prev_cy)
        frame_canvas = overlay_cursor(frame_canvas, pen_cursor, cur_x, cur_y)
        prev_cx, prev_cy = cur_x, cur_y

    out.write(cv2.cvtColor(frame_canvas, cv2.COLOR_RGB2BGR))

    if frame_num % (FPS * 3) == 0:
        elapsed = time.time() - render_start
        pct = frame_num / total_frames * 100
        print(f"  {pct:.0f}% ({frame_num}/{total_frames}) [{elapsed:.0f}s]", flush=True)
```

```
out.release()
render_time = time.time() - render_start
print(f"\n✅ Rendered {total_frames} frames in {render_time:.0f}s")
```

```
Background color: RGB(243, 239, 227)
🎬 Rendering multi-scene video (reveal: skeleton)
   Canvas: 1376x768
   Frames: 2085 (69.5s)
   Scenes: 4
   Transition: erase (1.0s)

Rendering...
  0% (0/2085) [0s]
  4% (90/2085) [3s]
  9% (180/2085) [6s]
  13% (270/2085) [9s]
  17% (360/2085) [11s]
  22% (450/2085) [13s]
  26% (540/2085) [16s]
  30% (630/2085) [19s]
  35% (720/2085) [21s]
  39% (810/2085) [24s]
  43% (900/2085) [27s]
  47% (990/2085) [30s]
  52% (1080/2085) [33s]
  56% (1170/2085) [35s]
  60% (1260/2085) [35s]
  65% (1350/2085) [36s]
  69% (1440/2085) [37s]
  73% (1530/2085) [37s]
  78% (1620/2085) [40s]
  82% (1710/2085) [42s]
  86% (1800/2085) [45s]
  91% (1890/2085) [47s]
  95% (1980/2085) [50s]
  99% (2070/2085) [51s]

✅ Rendered 2085 frames in 51s
```

Step 11: Merge Audio + Video

```
print("🔊 Merging audio and video...")

!ffmpeg -y -i {VIDEO_ONLY} -c:v libx264 -pix_fmt yuv420p -preset fast -crf 18 video_h264.mp4 -loglevel error
!ffmpeg -y -i video_h264.mp4 -i {AUDIO_FILE} -c:v copy -c:a aac -b:a 192k -shortest {FINAL_OUTPUT} -loglevel err

file_size = os.path.getsize(FINAL_OUTPUT) / (1024*1024)
print(f"\n✅ {FINAL_OUTPUT} ({file_size:.1f} MB)")

for f in [VIDEO_ONLY, 'video_h264.mp4']:
    if os.path.exists(f): os.remove(f)
```

```
🔊 Merging audio and video...

✅ whiteboard_video_final.mp4 (2.8 MB)
```

Start coding or generate with AI.

Step 12: Play & Download

```
with open(FINAL_OUTPUT, 'rb') as f:
    video_b64 = b64encode(f.read()).decode()

display(HTML(f"""
<h3>🎬 {video_plan['title']}</h3>
<video width="960" controls autoplay>
  <source src="data:video/mp4; base64,{video_b64}" type="video/mp4">
</video>
<p>{NUM_SCENES} scenes | {total_audio_duration:.1f}s | {file_size:.1f} MB</p>
"""))
```

🎬 **Cardiac Anatomy and Coronary Circulation Essentials**

Start coding or generate with AI.

0:11 / 1:07

```
from google.colab import files
files.download(FINAL_OUTPUT)

# Also save the plan for reference
with open('video_plan.json', 'w') as f:
    json.dump(video_plan, f, indent=2)
files.download('video_plan.json')

print("📩 Downloaded!")
```

📊 Configuration Reference

Video Length Guide

Target Length	NUM_SCENES	COMPONENTS_PER_SCENE	Approx Result
~30 seconds	1	4	Quick explainer
~1 minute	2	4	Short topic
~2 minutes	4	4	Standard educational
~3 minutes	6	4	Deep dive
~5 minutes	8	4-6	Full lecture segment
~10 minutes	12-15	4	Comprehensive video

Transition Styles

Style	Effect	Best For
`erase`	White wipe erases old scene	Classic whiteboard feel
`fade`	Old scene fades to white	Smoother, gentler
`instant`	Hard cut to white	Fast-paced videos

Tuning Tips

Setting	Effect
`REVEAL_RADIUS = 20`	Fine pen, slower reveal
`REVEAL_RADIUS = 50`	Thick brush, faster coverage
`TRANSITION_DURATION = 0.5`	Quick scene change
`TRANSITION_DURATION = 2.0`	Slow, dramatic erase
`FPS = 24`	Smaller file, slight stutter
`FPS = 30`	Smooth (recommended)